

# Orientation, Git, and how mobile apps are built

How the course runs, and four ways to build a mobile app

**Dinu-Ștefan Rusu**

FILS, Universitatea Politehnica București · Autumn 2026

# What you should leave with

---



## How this course runs

The fourteen lectures, the seven labs, how the grade is built, and what to bring to Lab 1.



## Four build strategies

Native, web, cross-platform in two flavors, and shared logic with native user interfaces.



## A working Git setup

A GitHub account and the loop you run every lab: branch, commit, push, pull request, review, merge.



## A choice you can defend

Why this course uses Flutter, what that choice costs, and when another strategy fits better.

PART 1 · ORIENTATION

# How this course works

Who teaches it, what you build, and how you are graded

# Who teaches this course

---

**Dinu-Ștefan Rusu**

dinu\_stefan.rusu@upb.ro

Microsoft Teams: course channel  
rusudinu.com

I build Flutter apps professionally. What you see in this course is the toolchain I use at work, not a teaching toy.

**45+**

Flutter apps shipped

**400k**

installs across them

---

**Ask during the lecture, not after it.** A question said out loud usually saves five other people the same confusion.

# What this course covers, and what comes next

---



## This course

Application Development for Mobile Devices. How to build the app: Dart and Flutter, state, networking, Firebase, authentication, release. Fourteen lectures and seven labs.



## Next semester

Mobile and Embedded Computing. What runs under the app: runtimes, concurrency, rendering cost, synchronization, remote procedure calls, and the embedded side.

---

**One course builds the app, the other explains the machine.** When a topic belongs to the second course, this one names it in a line and moves on.

# The fourteen lectures

---

---

WK

LECTURE

---

- 01 Orientation, Git, build strategies
  - 02 Dart, null safety, the toolchain
  - 03 Widgets and layout
  - 04 Async Dart, agent-assisted coding
  - 05 Debugging and state, part 1
  - 06 State management, part 2
  - 07 Networking: HTTP and JSON
- 

---

WK

LECTURE

---

- 08 Firebase, REST and GraphQL
  - 09 Observability and local storage
  - 10 Authentication, OAuth, App Check
  - 11 Routing, permissions, WebSockets
  - 12 AI in the app, LLMs and RAG
  - 13 User interface polish and testing
  - 14 Release, distribution, push
- 

Two hours every week. Each lecture builds on the one before it.

# The seven labs

---

---

| WEEK | LAB   | WHAT YOU BUILD                               |
|------|-------|----------------------------------------------|
| 2    | Lab 1 | Set-up, first repository, first pull request |
| 4    | Lab 2 | Dart classes, null-safe types, collections   |
| 6    | Lab 3 | The todo screen, widgets, and DevTools       |
| 8    | Lab 4 | JSON models and a network call with retry    |
| 10   | Lab 5 | The same app, rebuilt on BLoC                |
| 12   | Lab 6 | Authentication, routing, and a live screen   |
| 14   | Lab 7 | The practical lab test                       |

---

Labs run in even weeks. There is no semester project: the labs grow one app.

## How the grade is built

---

20 p

colloquium

20 p

midterm exam

30 p

lab assignments

30 p

practical lab test in Lab  
7, individual

---

**You pass with at least 50 points out of 100.** Most of the grade is decided in the lab, week by week, not in a single evening at the end.

# One repository, one branch per lab

---



## One repository

Each student keeps one GitHub repository named `admd-labs`. Every lab lands in it.



## One branch per lab

Work for Lab N goes on a branch named `lab-N`, created from `main` and pushed before the next lab.



## One pull request

Each lab ends with a pull request, merged into `main` so the next lab starts from it.

---

**Every lab lives in the repository.** History matters: small commits with real messages read better than one commit called final.

PART 2 · GIT AND GITHUB

# The workflow you will use

What Git is, and the loop you will run every lab

# Git, and the services that host it

---

Git is a version control system. It records every change ever made to your source code, and you can return to any of them.

Several people can work on the same codebase without overwriting each other.

Git runs on your machine. GitHub, GitLab and Bitbucket are services that host repositories and add collaboration on top: reviews, issues, automation.

## **One Git, many hosts**

The commands are identical everywhere. Only the website around them differs.

This course uses GitHub.

# Create a GitHub account before Lab 1

---

Sign up, free, at [github.com/signup](https://github.com/signup). Use a name you are happy to show a future employer.

Your commits are part of your grade. Every lab is read from your repository.

Turn on two-factor authentication. GitHub requires it for accounts that contribute code.

SHARED COURSE MATERIAL

[github.com/rusudinu/presentations](https://github.com/rusudinu/presentations)

YOUR OWN REPOSITORY

`admd-labs`, created in Lab 1

# Repositories and branches

---

A repository holds all the files of a project and every revision they have ever had.

It always has at least one branch. By convention that branch is called `main`.

A branch is a line of work: build a feature or fix a bug without touching anyone else's work.

A branch is always created from an existing branch, and later merged back into it.

## IN THIS COURSE

`main` is the finished state of your app. `lab-3` is the work in progress for Lab 3.

A repository is a Git concept, not a GitHub one. GitHub only hosts it.

# The workflow, command by command

---

```
git clone <repo-url>    # once
git checkout -b lab-1

# edit files, then:
git add .
git commit -m "Add todo screen"
git push -u origin lab-1

# open a pull request, get a review

git checkout main
git pull
```

## Run this loop once per lab.

Clone, branch, commit, push, review, merge, pull. Nothing else is needed for the whole semester.

`git status` answers most questions. Run it whenever you are unsure what Git is about to do.

# What a good commit message says

---

```
Add swipe to delete on the todo list  
Fix crash when the title is empty  
Move the API base URL into config
```

```
fix  
update stuff  
asdf
```

## **The first three are useful.**

They say what changed, in the imperative mood, in one line a reviewer can scan.

Keep the first line short, around seventy characters. One logical change per commit. If you need to explain why, put it in the body under a blank line.

---

**Write the message for the person reading it in six months.** That person is usually you, looking for the commit that broke something.

# Pull requests and review

---

A pull request proposes merging your branch into `main`, and shows the exact diff.

A reviewer reads it and either approves it or requests changes. Until the required approvals arrive, the pull request stays open.

You answer comments by pushing more commits to the same branch. The pull request updates itself.

## IN YOUR LABS

The description names what you built, what you could not finish, and any code an agent wrote for you.

---

**Open the pull request early, even when the work is unfinished.** Feedback on a half-built branch is cheap. Feedback on a finished one costs a rewrite.

# What happens after the merge

---



## 1 · Test

Continuous integration checks out the merged code, installs dependencies, runs the analyzer and the test suite.



## 2 · Build

If the tests pass, it builds the release artifacts: an Android bundle, an iOS archive, or a web bundle.



## 3 · Deliver

The artifact goes to testers or to a store track. Nobody uploads a build from a laptop.

Continuous integration, usually shortened to CI, is a service that runs those steps on a clean machine for every pull request and every merge.

---

**If it only builds on your laptop, it does not build.** A clean machine is the honest test of whether the repository holds everything the project needs.

PART 3 · STRATEGIES

# Four ways to build an app

Native, web, cross-platform in two flavors, and shared logic

# The baseline: native apps

---

Built for one operating system, with that platform's own software development kit and tools.

Best possible performance, and access to every capability the platform exposes on day one.

The platform's own widgets, so the app looks and behaves the way the operating system does.

## IOS

Swift and SwiftUI, built in Xcode, on a Mac

## ANDROID

Kotlin and Jetpack Compose, built in Android Studio

---

**Every other strategy sits on top of these two kits.** Cross-platform frameworks do not replace them: they generate or drive the same native application.

# Strategy 1: web apps and progressive web apps

---

## WHAT YOU GET

Opens from a link. No installation and no store review.

An update reaches every user the moment you deploy it.

One codebase for every platform that has a browser.

## WHAT YOU GIVE UP

Limited access to sensors and radios: Bluetooth, near-field communication, background location.

No reliable background execution, especially on iOS.

A browser performance ceiling, and a single engine on iOS.

A progressive web app, or PWA, is a web app that can be installed to the home screen, works offline, and is served over HTTPS.

## Strategy 2: React Native drives native widgets

---

### YOUR CODE

JavaScript or TypeScript, running in a JavaScript engine on the device

### THE LAYER UNDER IT

The JavaScript Interface, JSI, and the Fabric renderer, which call into the platform directly

### WHAT THE USER SEES

Real native widgets, one set per platform

The user interface is genuinely native, so each platform keeps its own look and behavior.

JavaScript can be updated over the air, without a new store release.

Every interaction crosses the boundary between JavaScript and native code.

---

**The old serialized bridge is gone.** Current React Native calls into native code directly, so older descriptions of a JavaScript bridge no longer apply.

## Strategy 2, other flavor: Flutter draws it

---

### YOUR CODE

Dart, compiled ahead of time to native machine code for release builds

### THE LAYER UNDER IT

Impeller, Flutter's renderer, which draws the whole user interface itself

### WHAT THE USER SEES

One user interface, identical everywhere

One rendering pipeline, so there are no per-platform layout surprises.

It reaches the operating system through platform channels and plugins.

Hot reload during development: a change appears in the running app in a second.

---

**React Native borrows the platform's widgets. Flutter brings its own.** That single difference explains almost every trade-off between the two.

## Strategy 3: Kotlin Multiplatform shares the logic

---

### SHARED KOTLIN MODULE

Business logic, networking, storage, models.  
Compiled natively for each platform.

### ANDROID APP

Its own user interface, in Jetpack Compose

### IOS APP

Its own user interface, in SwiftUI or in  
Compose Multiplatform

You choose how much to share, from one networking class to everything but the screens.

The `expect` and `actual` keywords bridge the parts that must differ per platform.

It is stable and in production at companies such as McDonald's, Netflix and Forbes.

---

**Flutter shares everything, including the pixels.** Kotlin Multiplatform shares exactly as much as you choose, and keeps each platform's native screens.

# The four side by side

| STRATEGY             | SHARED CODE                  | THE USER INTERFACE         | CHOOSE IT WHEN                 |
|----------------------|------------------------------|----------------------------|--------------------------------|
| Native               | none                         | the platform's own widgets | one platform, every capability |
| Web and PWA          | all of it                    | HTML and CSS in a browser  | reach without an install       |
| React Native         | logic and layout             | native widgets             | the team already writes React  |
| Flutter              | everything, drawing included | drawn by Flutter itself    | one team, identical screens    |
| Kotlin Multiplatform | as much as you choose        | native, written twice      | native apps already exist      |

# How to choose

---

---

## IF THIS DESCRIBES YOU

You need reach without installs, and light hardware needs

Your team lives in JavaScript and wants a native feel

You already ship two native apps and duplicate the logic

One small team, identical screens, fast iteration

## START WITH

Web or PWA

React Native

Kotlin Multiplatform

Flutter

---

**The constraint that decides is almost never technical.** Team skills, deadline and hardware access pick the strategy. Performance rarely does.

PART 4 · SETUP

# Flutter, and what to install

Why this course picked it, what it costs, and what to install

# Why this course uses Flutter

---



## One codebase

One student, working alone, can build a complete app for Android and iOS inside a two-hour lab.



## One renderer

What you see on the emulator is what a user sees on a phone. Less time lost to platform differences.



## Hot reload

A change shows up in the running app almost immediately, which is what makes the labs work.



## It is what I ship

The examples, the packages and the debugging habits come from apps that are in production now.

# What choosing Flutter costs

---

Binaries are larger than a native app of the same size, because the framework travels with the app.

Talking to the operating system goes through platform channels or a plugin, which is extra work and extra latency.

Dart is used mostly with Flutter, so the language itself transfers less than Kotlin or TypeScript would.

## **Say the cost out loud**

Every strategy costs something. An engineer who can only list the advantages of their tool has not finished evaluating it.

# Install this before Lab 1

---



## Flutter SDK

The software development kit, from flutter.dev. Add it to your PATH so `flutter` works in any terminal.



## Xcode, on a Mac

Only for building and running on iOS and the iOS Simulator. Skip it on Windows or Linux.

Install on the machine you bring to the lab. The downloads are large, so start them early.



## Android Studio

Needed for the Android toolchain and the Android emulator, even if you write code in another editor.



## Git and an account

Git on your machine, and the GitHub account from Part 2. Both are needed in the first lab.

# Check the install, and where things live

---

```
flutter doctor  
flutter create hello_admd  
cd hello_admd  
flutter run
```

## COURSE MATERIAL

[github.com/rusudinu/presentations](https://github.com/rusudinu/presentations)

## QUESTIONS

Microsoft Teams course channel, or  
in the lab

`flutter doctor` prints one line per toolchain component. Every line must be a check mark before Lab 1 starts.

# Three things to remember

---

1. Every lab lives in one repository. One repository, a branch per lab, a pull request per lab.
2. There are four ways to build a mobile app. Native, web, cross-platform, and shared logic with native screens.
3. Every strategy has a cost. This course uses Flutter, and states what that choice gives up.

## FURTHER READING

[docs.flutter.dev/get-started/install](https://docs.flutter.dev/get-started/install) · [docs.github.com/get-started](https://docs.github.com/get-started) · [kotlinlang.org](https://kotlinlang.org)  
[multiplatform](https://multiplatform.dev) · [reactnative.dev](https://reactnative.dev) architecture

# Questions?

dinu\_stefan.rusu@upb.ro · Microsoft Teams: course channel